SuperChat: Technical Report

One chat for flights, shopping and cinema · Phase 1 (mock data) · Reference date 2 Nov 2026 · Individual submission

1. Architecture

SuperChat is a FastAPI web app with a single-page chat UI (`static/index.html`). Every user message passes through the same pipeline; the user never picks a domain.

```
browser → POST /chat → engine.handle()
            1. router.route()      LLM call #1: JSON {language, route, is_followup, intents[]}   (fallback: rules.py)
            2. modules[domain].handle(state, action, args)   deterministic Python; one call per intent
            3. composer.compose()  LLM call #2: writes the reply from tool results only         (fallback: templates)
            4. state + history updated, reply returned with {category, domains, router}
```

Component	Responsibility
`router.py`, `rules.py`	Classify the message and extract slots (dates, prices, ids). Hybrid: LLM first, rule-based router as fallback.
`flights.py`, `shopping.py`, `cinema.py`	Domain logic: search, filter, sort, details, compare, cart, mock reservations with seat/stock checks. All filtering is done in Python, never by the LLM.
`composer.py`	Turns structured results into a short reply in the user's language. Sees only the results JSON, so it cannot add facts.
`state.py`	Per-domain and shared session state.
`repositories.py`	Data-access interfaces and JSON implementations.
`llm.py`	OpenAI-compatible client for NVIDIA NIM (`meta/llama-3.3-70b-instruct`), retry with back-off on 429/5xx, permanent errors switch the LLM off.

Design principle: the LLM **understands** (routing, slot extraction, wording); code **decides and computes** (what exists, what costs what, what is in stock). This is what satisfies the "no invented data" requirement (3.7).

2. Routing method and trade-offs

Method. A single LLM call receives the latest message, the last six turns, a compact summary of all three domain states, and a calendar for the reference date (so "Friday", "tonight", "Saturday" resolve to ISO dates). It returns one JSON object: `route` (*answer / ambiguous / out_of_scope*), `is_followup`, and a list of *intents*, each with a domain, an action from a fixed whitelist and arguments. The seven categories of the brief map as follows:

Brief category	How it is produced
Flights / Shopping / Cinema	One intent in the corresponding domain
Mixed	Two or more intents in different domains; all modules run, the composer writes one reply with a line per domain
Follow-up	`is_followup` plus ids resolved from state ("the second one" = item #2 of the last list of the active domain; "the cheaper one" = lowest price in the last list)
Ambiguous	`route=ambiguous`; one short clarifying question (also when a booking has no target in state, e.g. "Book two seats.")
Out of scope	`route=out_of_scope`; one-sentence polite refusal naming what the assistant can do

Validation. The output is sanitised: unknown domains/actions are dropped, ids not present in state are never trusted, and an "answer" with no valid intent is downgraded to "ambiguous". If the LLM is unreachable, rate-limited or returns invalid JSON (two attempts), `rules.py` produces the same schema from keyword lists (EN/AZ/RU), city/date parsers and ordinal patterns, so the pipeline after routing is identical.

Why this design, and the alternatives.

Option	Pros	Cons	Decision
Rules only	Free, instant, deterministic	Brittle for paraphrase, mixed intents and follow-ups; every language doubles the work	Kept as fallback
Embedding / trained classifier	Fast, cheap at scale	Needs labelled data; gives a label, not slots; weak on context-dependent follow-ups	Not used
Separate classifier call + separate extraction call	Easier to debug per step	Double latency and rate-limit use	Not used
One LLM call (classification + slots) + rules fallback	Handles mixed intents, follow-ups and three languages in one step; graceful degradation	Latency of ~2 LLM calls per message; free-tier rate limit (~40 req/min); LLM can still pick a wrong id	**Chosen**

3. Conversation state

- **Per domain.** Flights: last query, last results, selected flight, reservations. Shopping: last query, last results, list of earlier searches, selected product, cart. Cinema: last query, last movies/showtimes, selected movie and showtime, reservations.
- **Shared.** Language (updated every turn, so switching language mid-session works), stated preferences, `last_domain` (decides what "the second one" refers to).
- **Returning to a topic.** A domain's state is never cleared when another domain answers. "Back to the headphones" calls the `recall` action, which restores the earlier shopping search from `searches`; "back to the flights" does the same for flights.

- **Session history.** One transcript for the entire chat regardless of the domain that answered; the UI shows it all.
- **Mock reservations** decrement seats/stock in the repository (under a lock), so later queries see updated availability; sold-out items are flagged and cannot be reserved.

4. Data layer design (Phase 2 readiness)

Modules depend only on abstract repositories: `FlightRepository`, `ProductRepository`, `MovieRepository`, `CinemaRepository`, `ShowtimeRepository` (`all()`, `get(id)`, and `reserve_seats()` where relevant). No module imports `json` or touches files. `build_repositories()` selects the implementation from the `DATA_SOURCE` environment variable; today it returns the JSON implementations, and Phase 2 means writing `Api*` classes that implement the same interfaces (authentication, pagination, retries, caching, logging live inside them) and adding one branch to that factory. Domain modules, router, composer and state stay unchanged. Errors from any data source are caught in the engine per intent and turned into an `error` result that the composer explains in one sentence; stack traces are logged, never shown (3.7).

5. Testing summary

40 labelled conversations (tests/cases.py) cover all seven categories, the eight sample conversations from the brief, EN/AZ/RU, topic switches and "back to…" returns. 11 module tests (pytest) check filtering, sorting, comparison, cart totals, reservations and stock handling on the official mock data. Rules-only router: **38/40 = 95.0%** (errors: one cinema phrase "this week" classified out-of-scope; one Russian mixed request routed to shopping only). LLM-mode results are reported in `TEST_LOG.md` when run with an API key.. Details and error analysis are in `TEST_LOG.md`.

6. Known limitations

- Hosted-LLM dependency: quality, latency and the free-tier rate limit depend on NVIDIA NIM. Without a key the app runs rules-only with English replies and simpler routing (mixed intents and AZ/RU phrasing are weakest there).
- Two LLM calls per message add latency; the composer is skipped only when the LLM is unavailable.
- Sessions and seat/stock changes are in memory: lost on restart, not shared across processes. No authentication.
- Routing errors can still occur on long, multi-clause messages; the module layer prevents invented data but cannot fix a wrongly chosen item.
- Round-trip flights, visual search and voice input are not implemented (optional/bonus); multilingual support (AZ/RU/EN) is implemented through the router and composer.
- Reference date is fixed to 2026-11-02 via `config.REF_DATE`; production needs the real clock and time zones.

7. Sources and licences

FastAPI and Uvicorn (MIT/BSD), OpenAI Python SDK (Apache-2.0) used as an OpenAI-compatible client, httpx (BSD), pytest (MIT). LLM: Meta Llama 3.3 70B Instruct (Llama 3.3 Community License) via NVIDIA NIM free API. Official mock dataset supplied by the organisers. No personal or payment data is used.